



AGH

AGH UNIVERSITY OF KRAKOW

**Otoczka wypukła dla zbioru punktów w
przestrzeni dwuwymiarowej**

Temat 4, Grupa 1

Autorzy: Mateusz Wójcik, Błażej Naziemiec

5 stycznia 2025

Wydział Informatyki

Spis treści

1	Wstęp	3
2	Część techniczna	3
2.1	Struktura plików projektu	3
2.2	Opis struktur używanych w części głównej	4
2.3	Wykorzystane biblioteki i ich zastosowanie	4
3	Część użytkownika	5
4	Sprawozdanie	6
4.1	Opis problemu	6
4.2	Struktura danych	6
4.3	Algorytm przyrostowy	7
4.3.1	Działanie algorytmu	7
4.3.2	Wizualizacja przebiegu algorytmu	7
4.3.3	Analiza złożoności obliczeniowej	7
4.4	Algorytm wyznaczania górnej i dolnej otoczki wypukłej	8
4.4.1	Działanie algorytmu	8
4.4.2	Wizualizacja przebiegu algorytmu	8
4.4.3	Analiza złożoności obliczeniowej	8
4.5	Algorytm Quickhull	9
4.5.1	Działanie algorytmu	9
4.5.2	Wizualizacja przebiegu algorytmu	9
4.5.3	Analiza złożoności obliczeniowej	9
4.6	Algorytm dziel i rządź	10
4.6.1	Działanie algorytmu	10
4.6.2	Wizualizacja przebiegu algorytmu	10
4.6.3	Analiza złożoności obliczeniowej	11
4.7	Algorytm Chan’a	12
4.7.1	Działanie algorytmu	12
4.7.2	Analiza złożoności obliczeniowej	12
4.7.3	Wizualizacja przebiegu algorytmu	13
4.8	Algorytm Grahama	14
4.8.1	Działanie algorytmu	14
4.8.2	Wizualizacja przebiegu algorytmu	14
4.8.3	Analiza złożoności obliczeniowej	15
4.9	Algorytm Jarvisa	16
4.9.1	Działanie algorytmu	16
4.9.2	Analiza złożoności obliczeniowej	16
4.9.3	Wizualizacja przebiegu algorytmu	16
4.10	Testy algorytmów	17
4.10.1	Zestaw 1: Losowy zestaw punktów	17
4.10.2	Zestaw 2: Zbiór punktów tworzących otoczkę 4-8 elementową	18
4.10.3	Zestaw 3: Zbiór punktów tworzących otoczkę 4 elementową	18
4.10.4	Zestaw 4: Zbiór punktów na planie okręgu	19
4.11	Wyniki czasowe działania algorytmów	21
4.11.1	Zestaw 1	21
4.11.2	Zestaw 2	22
4.11.3	Zestaw 3	23
4.11.4	Zestaw 4	24
4.12	Wnioski i podsumowanie	25
4.12.1	Zbiór 1: Losowo rozmieszczone punkty	25

4.12.2	Zbiór 2: Zbiór z otoczką z 8 punktami	25
4.12.3	Zbiór 3: Zbiór z otoczką z 4 punktami	25
4.12.4	Zbiór 4: Wszystkie punkty należą do otoczki	25
4.12.5	Podsumowanie	26
5	Bibliografia	27

1 Wstęp

W ramach projektu zaimplementowano kilka algorytmów wyznaczania otoczki wypukłej, czyli najmniejszego wielokąta wypukłego zawierającego wszystkie punkty danego zbioru. Celem projektu była zarówno implementacja tych algorytmów, jak i ich szczegółowa analiza pod kątem efektywności obliczeniowej, z uwzględnieniem złożoności czasowej oraz identyfikacją ich mocnych i słabych stron w różnych scenariuszach zastosowań.

2 Część techniczna

Implementacja algorytmów została przeprowadzona w języku Python 3.11.9, z wykorzystaniem zarówno wbudowanych, jak i zewnętrznych bibliotek. Pliki projektu zapisano w formacie .ipynb, co umożliwia interaktywne programowanie i analizę wyników w środowisku Jupyter Notebook. Testy przeprowadzono na komputerze z systemem operacyjnym Microsoft Windows 11, wyposażonym w procesor Intel Core i5-1235U.

2.1 Struktura plików projektu

W celu zachowania czytelności, modularności oraz optymalizacji kodu, projekt został podzielony na odrębne moduły, z których każdy pełni ściśle określoną funkcję. Struktura plików projektu przedstawia się następująco:

- **interface.ipynb** Moduł zawierający kluczowe klasy wykorzystywane w projekcie, w tym:
 - **Point** – klasa reprezentująca punkty w przestrzeni dwuwymiarowej,
 - **Segment** – klasa modelująca odcinki łączące dwa punkty,
 - **PolygonDrawer** klasa umożliwiająca graficzne definiowanie zbioru punktów,
 - **ConvexHullVisualizer** klasa odpowiedzialna za wizualizację przebiegu algorytmów w czasie rzeczywistym.
- **install.ipynb** Plik odpowiedzialny za importowanie niezbędnych bibliotek zewnętrznych. Pozwala na automatyczne instalowanie brakujących zależności, co upraszcza konfigurację środowiska dla użytkownika.
- **tests.ipynb** Moduł przeznaczony do testowania poprawności zaimplementowanych algorytmów. Zawiera zarówno testy jednostkowe sprawdzające poprawność wyników, jak i testy czasowe służące do analizy wydajności poszczególnych rozwiązań.
- **Pliki implementacyjne algorytmów** Każdy z algorytmów został zaimplementowany w osobnym pliku, co zwiększa przejrzystość kodu i umożliwia jego łatwą rozbudowę. Pliki te zawierają:
 - Implementację algorytmu w wersji bazowej (bez wizualizacji),
 - Możliwość definiowania zbiorów punktów w sposób graficzny,
 - Testowanie poprawności algorytmu.
 - Wizualizację wyników oraz przebiegu działania algorytmów na zadanych danych.

Takie podejście modularne ułatwia zarządzanie kodem oraz wprowadzanie zmian. Każdy moduł został zaprojektowany tak, aby spełniał pojedynczą odpowiedzialność. Dzięki temu struktura projektu jest intuicyjna i przejrzysta. Przy omówieniu każdego algorytmu (podpunkty 4.3 - 4.9) została podana informacja, który plik odpowiada implementacji konkretnego algorytmu.

2.2 Opis struktur używanych w części głównej

- **Klasa Point** reprezentuje punkt w przestrzeni 2D, przechowujący współrzędne x i y . Konstruktor `__init__` inicjalizuje obiekt punktu, przypisując wartości współrzędnych. Metoda `__repr__` zapewnia czytelne wyświetlanie punktu w formacie `Point(x, y)`.
- **Klasa Segment** reprezentuje odcinek w przestrzeni 2D, łączący dwa punkty. Konstruktor `__init__` inicjalizuje obiekt odcinka, przypisując punkty początkowy (`first_point`) i końcowy (`second_point`). Metoda `reversed` zwraca nowy obiekt odcinka z zamienionymi miejscami punktami początkowym i końcowym.
- **Klasa PolygonDrawer** umożliwia interaktywne rysowanie i zapisywanie współrzędnych punktów w przestrzeni 2D. Konstruktor `__init__` inicjalizuje obiekt, tworząc okno graficzne oraz przyciskiem `Wyczyść`, który usuwa wszystkie narysowane punkty z pamięci.

Działanie klasy obejmuje:

- Metodę `onclick`, która reaguje na pojedyncze kliknięcia myszą w obszarze rysowania, automatycznie dodając nowy punkt do listy i aktualizując rysunek.
- Metodę `draw_polygon`, odpowiedzialną za rysowanie wszystkich aktualnie dodanych punktów na wykresie.
- Metodę `clear_polygon`, która usuwa wszystkie punkty i resetuje wykres.

Ważnym elementem przy dodawaniu nowego punktu myszką jest fakt, iż aby dodać punkt w danym miejscu należy kliknąć w dane miejsce tylko raz.

- **Klasa ConvexHullVisualizer** służy do wizualizacji algorytmów wyznaczania otoczki wypukłej. Umożliwia tworzenie animacji obrazujących proces budowania otoczki oraz zarządzanie klatkami (stanami pośrednimi) w trakcie obliczeń dla różnych algorytmów.
 - `__init__` Konstruktor inicjalizuje obiekt, ustawiając domyślną prędkość animacji (`speed`) oraz listę klatek (`frames`), w której przechowywane są kolejne stany wizualizacji.
 - `add_frame(points, hulls_closed, hulls_not_closed, segments, current_point)` Dodaje nową klatkę wizualizacji, przechowując informacje o punktach (`points`), zamkniętych i otwartych częściach otoczek w postaci list (`hulls_closed`, `hulls_not_closed`), odcinkach (`segments`) oraz bieżącym punkcie przetwarzania (`current_point`).
 - `create_animation()` Tworzy animację na podstawie zgromadzonych klatek. Funkcja zawiera:
 - * `sort_points_by_angle` — Sortuje punkty według kąta, co jest używane do poprawnego wyświetlania zamkniętych otoczek.
 - * `update(frame)` — Aktualizuje rysunek dla każdej klatki, rysując punkty, odcinki, zamknięte i otwarte otoczki oraz bieżący punkt przetwarzania.

Animacja wyświetla proces przebiegu algorytmu i zwraca wynik w formacie HTML, który można osadzić w notatniku Jupyter.

2.3 Wykorzystane biblioteki i ich zastosowanie

W projekcie wykorzystano następujące biblioteki:

- **matplotlib** Biblioteka odpowiedzialna za tworzenie wizualizacji danych. W projekcie jest używana do rysowania punktów, odcinków oraz wizualizacji przebiegu algorytmów wyznaczających otoczkę wypukłą.
- **IPython** Używana do integracji z Jupyter Notebook w celu wyświetlania animacji bezpośrednio w środowisku notebooka. Funkcja `HTML` z tej biblioteki umożliwia osadzanie animacji generowanych przez `matplotlib`.

- **ipywidgets** Biblioteka pozwalająca na tworzenie interaktywnych widżetów w Jupyter Notebook.
- **math** Biblioteka używana do obliczania kątów za pomocą `math.atan2` w celu sortowania punktów według kąta w algorytmie wyznaczania otoczki wypukłej w klasie `ConvexHullVisualizer`, co pozwala na poprawne przedstawienie zbioru punktów tworzących otoczkę.
- **numpy** Biblioteka używana do obliczeń numerycznych, w tym funkcji `np.arccos` czy `np.arctan2`, służącej do precyzyjnego wyznaczania kątów między punktami.

3 Część użytkownika

Aby uruchomić algorytm, należy otworzyć odpowiedni plik i wybrać opcję `Run All`, co umożliwia graficzne wprowadzenie dowolnego zbioru punktów. Następnie należy przejść do sekcji odpowiedzialnej za wizualizację i uruchomić ją, aby wyświetlić przebieg algorytmu. Testy poprawności można sprawdzić, uruchamiając kod w sekcji testowej i analizując uzyskane wyniki.

Po uruchomieniu sekcji testowej użytkownik otrzymuje informację o liczbie punktów w przypadku testowym oraz o poprawności wyników algorytmu. Uruchomienie wizualizacji pozwala na bezpośrednie wyświetlenie przebiegu algorytmu w Jupyterze. Z kolei uruchomienie podstawowego algorytmu zwraca użytkownikowi listę punktów należących do otoczki wypukłej.

4 Sprawozdanie

W tej części zostanie opisany sposób przygotowanego rozwiązania. Dokładnie zostanie opisany problem otoczki wypukłej oraz użyte struktury danych. W dalszej części zostanie opisany dokładnie każdy algorytm - sposób działania, z jakich funkcji korzysta oraz za co dokładnie są one odpowiedzialne.

4.1 Opis problemu

Celem projektu jest implementacja, wizualizacja i analiza algorytmów wyznaczania otoczki wypukłej dla zbioru punktów w przestrzeni dwuwymiarowej. Otoczka wypukła to najmniejszy wielokąt wypukły obejmujący wszystkie punkty z danego zbioru. W ramach zadania zaimplementowano algorytmy: przyrostowy, górna i dolna otoczka, Quickhull, dziel i rządź oraz Chan’a, a także porównano je z algorytmami Grahama i Jarvisa. Analiza obejmuje porównanie efektywności algorytmów pod względem złożoności czasowej, praktycznego czasu wykonania oraz analizy ich mocnych i słabych strony w zależności od charakterystyki zbiorów testowych, takich jak liczba, rozmieszczenie i losowość punktów.

4.2 Struktura danych

Każdy z przygotowanych algorytmów wykorzystuje wspomnianą w podpunkcie 2.2 klasę `Point`. Służy ona jako odzwierciedlenie punktu umieszczonego w układzie kartezjańskim. Użycie takiej struktury danych pozwala pisać oraz czytać kod algorytmu w bardzo intuicyjny sposób, a także zapewnia optymalne rozwiązania czasowe. Jako argument algorytmy przyjmują listę z elementami właśnie klasy `Point`. Wyjątkiem jest funkcja `chan_with_m`, która oprócz listy elementów `Point` przyjmuje m - aproksymację liczby punktów końcowej otoczki wypukłej. Jest ona potrzebna do utrzymania prawidłowej złożoności algorytmu (więcej na temat algorytmu, funkcji oraz jego złożoności w podpunkcie 4.7). Jako dane wyjściowe algorytm zwraca listę elementów klasy `Point`, które oznaczają kolejne wierzchołki finalnej otoczki wypukłej. Dodatkowo elementy te znajdują się w posortowanej kolejności. Kolejność ta opisuje kolejne punkty końcowej otoczki. Oznacza to, że każda dowolna para sąsiadów odpowiada końcom kolejnych odcinków finalnej otoczki. Dotyczy to również ostatniego oraz pierwszego elementu listy, które tworzą ostatni odcinek otoczki. Dzięki temu rozwiązaniu w łatwy sposób użytkownik może odtworzyć otoczkę i sprawdzić graficzne rozwiązanie we własnym programie.

4.3 Algorytm przyrostowy

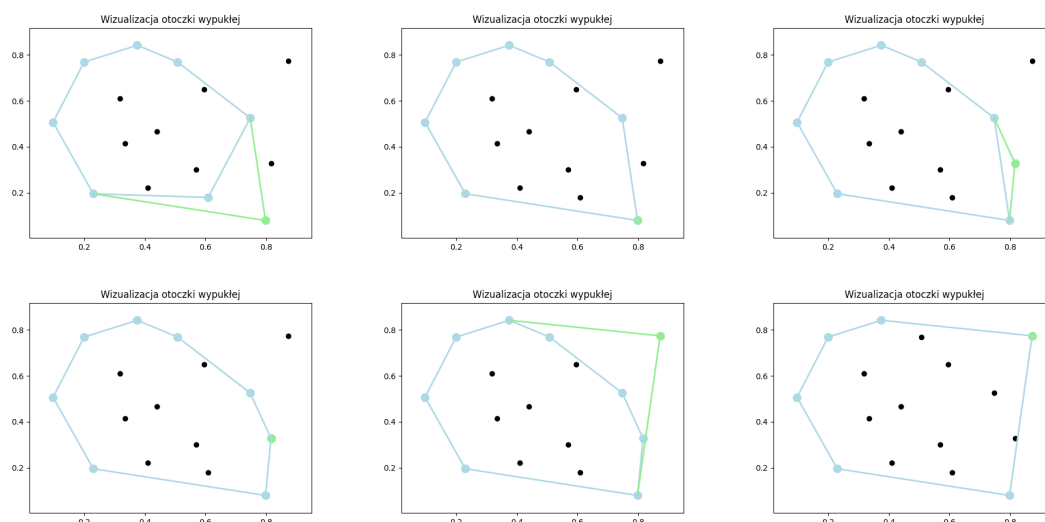
Nazwa pliku z implementacją: `incremental.ipynb`

4.3.1 Działanie algorytmu

Algorytm inkrementalny wyznaczania otoczki wypukłej opiera się na iteracyjnym dodawaniu kolejnych punktów do aktualnej otoczki. Na początku punkty są sortowane rosnąco według współrzędnych (x, y) , co pozwala na ich przetwarzanie w ustalonej kolejności. Następnie, na podstawie pierwszych dwóch punktów, tworzona jest początkowa otoczka, które uzyskujemy dzięki posortowaniu zbioru punktów. Dla każdego kolejnego punktu algorytm wyznacza jego pozycję względem górnej i dolnej części otoczki, iteracyjnie określając punkty, z którymi nowy punkt tworzy minimalny i maksymalny kąt. Wykorzystywana jest do tego funkcja `orient`. Zaktualizowana otoczka powstaje przez połączenie punktów leżących pomiędzy wyznaczonymi indeksami oraz dodanie nowego punktu.

Funkcja `orient` służy do określenia orientacji trójki punktów na podstawie wyznacznika macierzy współrzędnych, co pozwala rozróżnić konfiguracje współliniowe, zgodne lub przeciwnie do ruchu wskazówek zegara.

4.3.2 Wizualizacja przebiegu algorytmu



Rysunek 1: Fragment przebiegu algorytmu przyrostowego dla losowego zbioru punktów

4.3.3 Analiza złożoności obliczeniowej

Złożoność obliczeniowa algorytmu przyrostowego wyznaczania otoczki wypukłej wynosi $O(n \log n)$, gdzie n to liczba punktów w zbiorze wejściowym. Kluczowym krokiem jest wstępne sortowanie punktów leksykograficznie, co wymaga $O(n \log n)$ operacji. Następnie, dla każdego punktu, algorytm znajduje tzw. wierzchołki przejściowe na otoczce ($O(n)$ w najgorszym przypadku) i aktualizuje zbiór punktów tworzących otoczkę. Dzięki temu, że punkty przetworzone są pomijane w dalszych obliczeniach, złożoność iteracyjnej części algorytmu jest amortyzowana. W efekcie całkowita złożoność algorytmu pozostaje rzędu $O(n \log n)$.

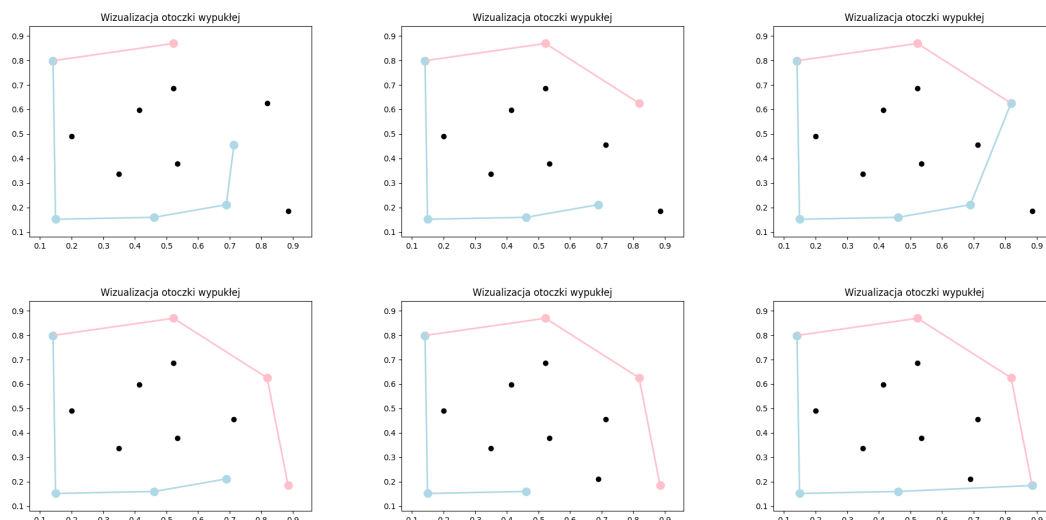
4.4 Algorytm wyznaczania górnej i dolnej otoczki wypukłej

Nazwa pliku z implementacją: `lower-upper-hull.ipynb`

4.4.1 Działanie algorytmu

Algorytm wyznaczania górnej i dolnej otoczki wypukłej dla zbioru punktów na płaszczyźnie rozpoczyna się od posortowania punktów rosnąco według współrzędnych (x, y) . Następnie iteracyjnie budowana jest górna i dolna część otoczki. W procesie budowy, do każdej z list (`upper_hull` oraz `lower_hull`) dodawane są kolejne punkty, a ich wzajemne położenie względem prostej wyznaczonej przez dwa ostatnie punkty otoczki jest sprawdzane za pomocą funkcji określającej orientację punktu (`position`). Jeśli nowy punkt powoduje naruszenie warunku wypukłości, ostatni punkt na liście jest usuwany. Po przetworzeniu wszystkich punktów ostatnie elementy obu list są usuwane w celu uniknięcia ich powtórzenia, a dolna otoczka jest odwracana i łączona z górną. Algorytm, dzięki zastosowaniu sortowania o złożoności $O(n \log n)$ oraz liniowego przetwarzania punktów, osiąga całkowitą złożoność $O(n \log n)$. Wynikiem jest lista punktów reprezentujących otoczkę wypukłą.

4.4.2 Wizualizacja przebiegu algorytmu



Rysunek 2: Fragment przebiegu algorytmu dolnej i górnej otoczki dla losowego zbioru punktów

4.4.3 Analiza złożoności obliczeniowej

Algorytm wyznaczania otoczki wypukłej składa się z dwóch etapów: sortowania punktów o złożoności $O(n \log n)$ oraz iteracyjnego budowania otoczki, które przebiega w czasie $O(n)$ dzięki każdorazowemu dodaniu lub usunięciu punktu dokładnie raz. Dominującym etapem jest sortowanie, co sprawia, że całkowita złożoność czasowa algorytmu wynosi $O(n \log n)$. Złożoność pamięciowa jest liniowa, $O(n)$, ponieważ algorytm wymaga jedynie przechowywania wejściowego zbioru punktów oraz list reprezentujących otoczkę. Warto zauważyć, że całkowita złożoność algorytmu nie zależy od liczby punktów tworzących otoczkę wypukłą, co jest zarówno zaletą, ponieważ zapewnia stabilną wydajność niezależnie od rozkładu punktów, jak i wadą, ponieważ nie wykorzystuje potencjalnej rzadkości punktów tworzących otoczkę w stosunku do liczności całego zbioru.

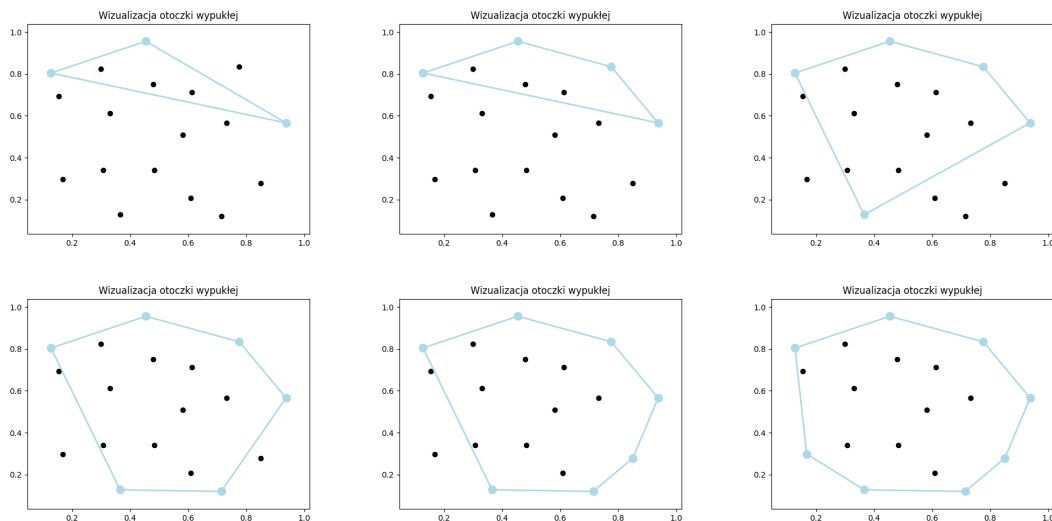
4.5 Algorytm Quickhull

Nazwa pliku z implementacją: `quickhull.ipynb`

4.5.1 Działanie algorytmu

Algorytm QuickHull wyznacza otoczkę wypukłą zbioru punktów w przestrzeni dwuwymiarowej, bazującą na podejściu "dziel i rządź". Proces rozpoczyna się od znalezienia punktów skrajnych, tj. punktu o minimalnej i maksymalnej współrzędnej x , które tworzą linię bazową. Następnie zbiór punktów dzieli się na dwie części – punkty znajdujące się powyżej i poniżej tej linii. Podział ten jest uzyskiwany za pomocą funkcji *position*, która zwraca informację po jakiej stronie prostej znajduje się punkt. Dla każdego z tych podzbiorów algorytm rekurencyjnie identyfikuje punkt najbardziej oddalony od linii bazowej, przy pomocy funkcji *relative_distance*, który należy do otoczki wypukłej, a zbiór punktów jest dalej dzielony na mniejsze części w zależności od lokalizacji punktów względem nowo wyznaczonej linii. Proces ten powtarza się, aż wszystkie punkty zostaną uwzględnione w obrysie otoczki. Złożoność średnia wynosi $O(n \log n)$, jednakże w przypadku gdy wszystkie punkty zbioru należą do otoczki złożoność algorytmu wyniesie $O(n^2)$.

4.5.2 Wizualizacja przebiegu algorytmu



Rysunek 3: Fragment przebiegu algorytmu Quickhull dla losowo zadanego zbioru punktów

4.5.3 Analiza złożoności obliczeniowej

Algorytm QuickHull ma złożoność $O(n \log n)$ w najlepszym przypadku, gdy otoczką wypukłą składa się z niewielkiej liczby punktów, np. czterech, a większość punktów leży wewnątrz niej. W najgorszym przypadku, gdy wszystkie punkty należą do otoczki (np. w przypadku punktów rozmieszczonych na okręgu), złożoność wzrasta do $O(n^2)$, ponieważ algorytm musi przetworzyć każdy punkt na każdym etapie rekurencji.

4.6 Algorytm dziel i rządź

Nazwa pliku z implementacją: `divide-and-conquer.ipynb`

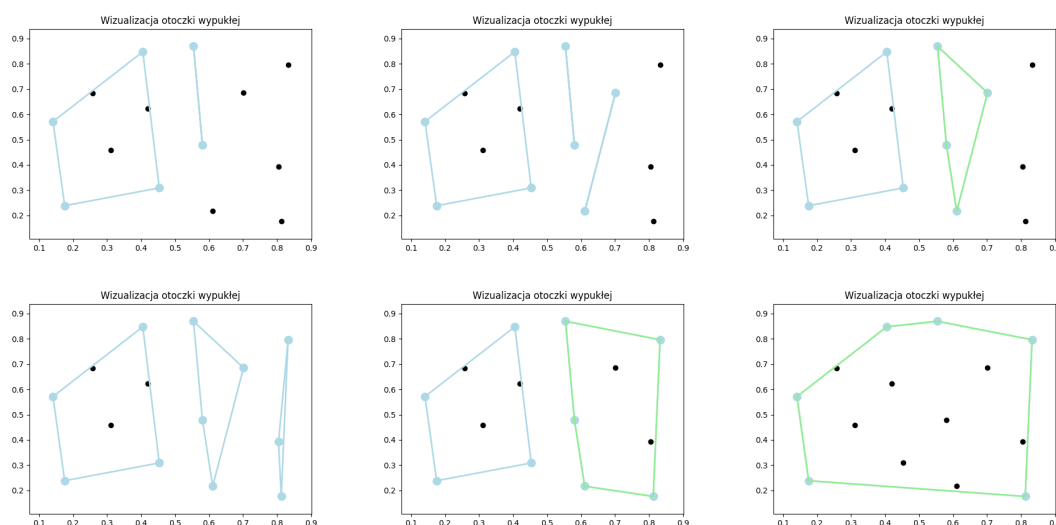
4.6.1 Działanie algorytmu

Algorytm dziel i rządź do wyznaczenia otoczki wypukłej wykorzystuje taktykę używaną np. w algorytmie MergeSort, czyli "divide-and-conquer". Na początku zbiór punktów jest sortowany rosnąco względem ich współrzędnych x-owych oraz y-owych.

Następnie punkty te dzielone są rekurencyjnie na mniejsze, maksymalnie trzy elementowe grupy za pomocą funkcji *divide_points*. Dzięki tak niewielkiej liczbie punktów oraz funkcji orient, zwracającą informacje, czy 3 punkty tworzą lewo/prawoskrętny kąt lub współliniowe, jesteśmy w stanie w złożoności $O(1)$ wyznaczyć otoczkę tego podzbioru i zwrócić posortowaną kolejność punktów w otoczce wypukłej. Odpowiedzialna jest za to funkcja *set_hulls*. Po wyznaczeniu tych otoczek, podobnie jak w algorytmie MergeSort scalamy dwie otoczki w jedną wykorzystując funkcję *merge_hulls*.

Na jej początku za pomocą funkcji *upper_section* wyznaczamy indeksy punktów, które będą wierzchołkami odcinka łączącego otoczki na górze. Analogicznie wyznaczamy indeksy dla punktów wierzchołków dolnego odcinka z użyciem funkcji *lower_section*. Obie te funkcje w najgorszym przypadku wyznaczają odpowiednie punkty w złożoności $O(n)$. Na początku za pomocą funkcji *max_x_position* oraz *min_x_position* wyznaczane są odpowiednio punkty z największą współrzędną x-ową w lewej otoczce oraz najmniejszą współrzędną x-ową w prawej otoczce. Następnie dopóki oba punkty nie tworzą stycznej do lewej i prawej otoczki, wybieramy dolnych/górnych sąsiadów aktualnie rozpatrywanych punktów w obu otoczkach. To, czy wybieramy dolnego, czy górnego sąsiada punktu zależy, czy szukamy aktualnie dolnej czy górnej stycznej do obu otoczek (dolni sąsiedzi w przypadku dolnej stycznej, górni w przypadku górnej stycznej). Po wyznaczeniu odpowiednich punktów w funkcji *merge_hulls* w złożoności $O(n)$ łączymy obie otoczki tworząc nową, większą otoczkę. Działanie to wykonujemy na każdym poziomie rekurencji. Ostatecznie otrzymujemy finalną otoczkę dla wszystkich punktów zbioru.

4.6.2 Wizualizacja przebiegu algorytmu



Rysunek 4: Fragment przebiegu algorytmu dziel i rządź dla losowo zadanego zbioru punktów

4.6.3 Analiza złożoności obliczeniowej

Algorytm dziel i rządź dla wyznaczania otoczki wypukłej ma złożoność czasową $O(n \log n)$, gdzie n to liczba punktów w zbiorze wejściowym. Proces rozpoczyna się posortowaniem punktów rosnąco względem ich współrzędnych x-owych oraz y-owych w złożoności $O(n \log n)$, a następnie podziału zbioru punktów na dwa podzbiory, co wymaga $O(\log n)$ poziomów rekurencji, przy czym na każdym poziomie punkty są przetwarzane w $O(n)$. Łączenie otoczek obu podzbiorów odbywa się w czasie liniowym. Złożoność pamięciowa wynosi $O(n)$, ponieważ dodatkowe struktury danych są ograniczone do przechowywania punktów i wynikowych otoczek na każdym poziomie rekurencji.

4.7 Algorytm Chan’a

Nazwa pliku z implementacją: `chan.ipynb`

4.7.1 Działanie algorytmu

Algorytm Chan’a łączy w sobie zarówno algorytm Grahama, jak i Jarvisa. Kluczową rzeczą w osiągnięciu poprawnej złożoności tego algorytmu jest wyznaczenie m - aproksymacja liczby wierzchołków otoczki wypukłej.

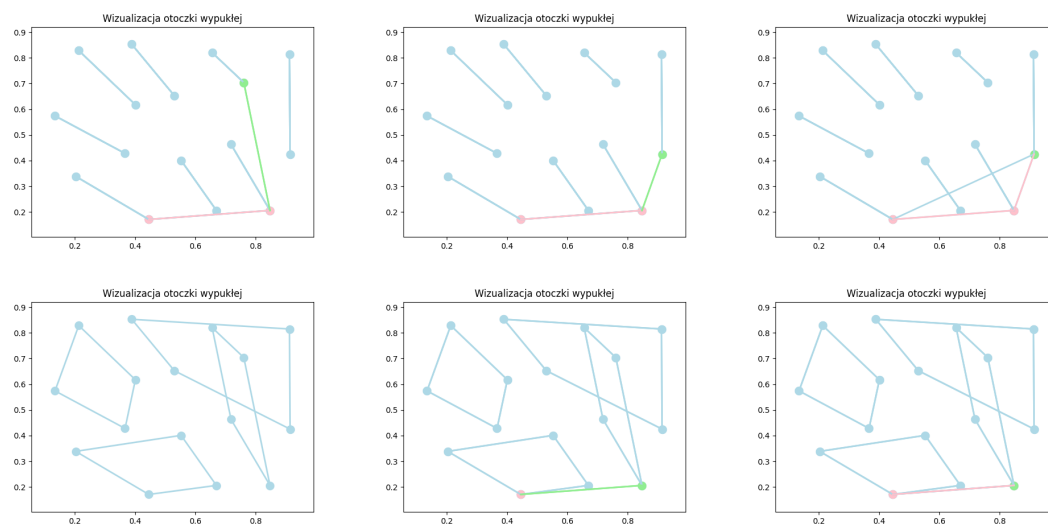
Cały algorytm realizuje funkcja `PartialHull` wraz z funkcjami pomocniczymi. Początkowo zbiór wszystkich punktów dzielimy na r podgrup, gdzie $r = \lceil n/m \rceil$, każda z nich posiadająca maksymalnie m punktów (dopuszczalny jest przypadek, gdy w ostatniej podgrupie znajdzie się mniej punktów). Odpowiedzialna za to funkcja `divide_all_points` zwraca listę list zawierających punkty należące do danej podgrupy. Następnie dla każdej z podgrup wyznaczamy otoczkę wypukłą za pomocą algorytmu Grahama (funkcja `graham_algorithm`). Dzięki takiemu działaniu dostajemy posortowane w kierunku przeciwnym do ruchu wskazówek zegara punkty oraz dodatkowo wyeliminowaliśmy kilka punktów. W kolejnym kroku używając funkcji `find_lowest_point` przeszukując wszystkie wyznaczone otoczki znajdujemy punkt, który ma najmniejszą współrzędną y (w przypadku kilku punktów o tej samej współrzędnej y wybierany jest punkt z najmniejszą współrzędną x). W następnym kroku podobnie niczym w algorytmie Jarvisa wyznaczamy styczne do ostatniego wyznaczonego odcinka, który tworzy finalną otoczkę. Jednak w tym przypadku wyznaczamy ją dla każdej otoczki i wybieramy ostatecznie odcinek, który tworzy najmniejszy kąt. W każdej z otoczce jesteśmy w stanie wyznaczyć skrajny punkt, który będzie tworzył najmniejszy kąt w złożoności $O(\log n)$, ponieważ punkty w każdej otoczce są posortowane i wyszukiwanie binarne znajdzie w takiej złożoności szukany punkt. Funkcją odpowiedzialną za to jest `find_next_point_in_hull`. Operację tą powtarzamy m razy, dopóki nie okaże się, że punktem stycznym do otoczki jest punkt początkowy. Wtedy to wyznaczona została cała otoczka wypukła zbioru. Jeśli nie uda się wyznaczyć końcowej otoczki, oznacza to, że podana wartość m jest za mała i finalna otoczka posiada więcej punktów.

W ramach rozwiązania przygotowano dwie funkcje: `chan` oraz `chan_with_m`. Funkcja `chan` jako argument przyjmuje zbiór punktów oraz sama próbuje wyznaczyć liczbę punktów otoczki. W tym celu na początku wyznacza wartość $m = \min(2^t, n)$, gdzie $t = 1, 2, \dots$. Dla każdej kolejnej wartości m algorytm próbuje wyznaczyć otoczkę wypukłą z zadaną liczbą wierzchołków. Funkcja `chan_with_m` jako argumenty przyjmuje zbiór punktów oraz wartość m , która jest aproksymacją użytkownika, ile punktów tworzy końcową otoczkę. Jeśli podana wartość m jest za mała, program zwraca informację "Podana liczba wierzchołków otoczki jest za mała". W obu algorytmach w przypadku wyznaczenia finalnej otoczki zwracana jest lista punktów tworzących ją.

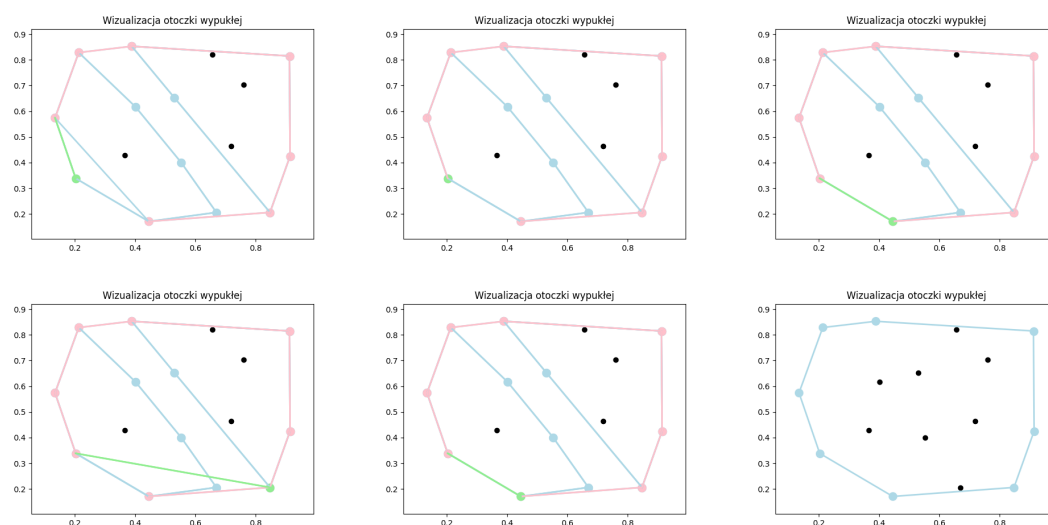
4.7.2 Analiza złożoności obliczeniowej

Złożoność algorytmu Chana wynosi $O(n \log m)$. Jednak osiągnięcie takiej złożoności jest możliwe, kiedy $m = h$, gdzie h to liczba punktów w otoczce końcowej. W innym przypadku złożoność tego algorytmu wynosi $O(n + (hn/m) \log m)$. Podzielenie punktów na r grup to złożoność $O(n)$, a wyznaczenie otoczki wypukłej dla każdej m -elementowej podgrupy wynosi $O(rm \log m) = O(n \log m)$. Znalezienie za pomocą wyszukiwania binarnego skutecznego punktu wynosi $O(\log m)$. Końcowe użycie sposobu z algorytmu Jarvisa dla r otoczek to złożoność $O((hn/m) \log m)$.

4.7.3 Wizualizacja przebiegu algorytmu



Rysunek 5: Konstruowanie otoczek w algorytmie Chan'a



Rysunek 6: Wyznaczanie końcowej otoczki wypukłej w algorytmie Chan'a

4.8 Algorytm Grahama

Nazwa pliku z implementacją: `graham.ipynb`

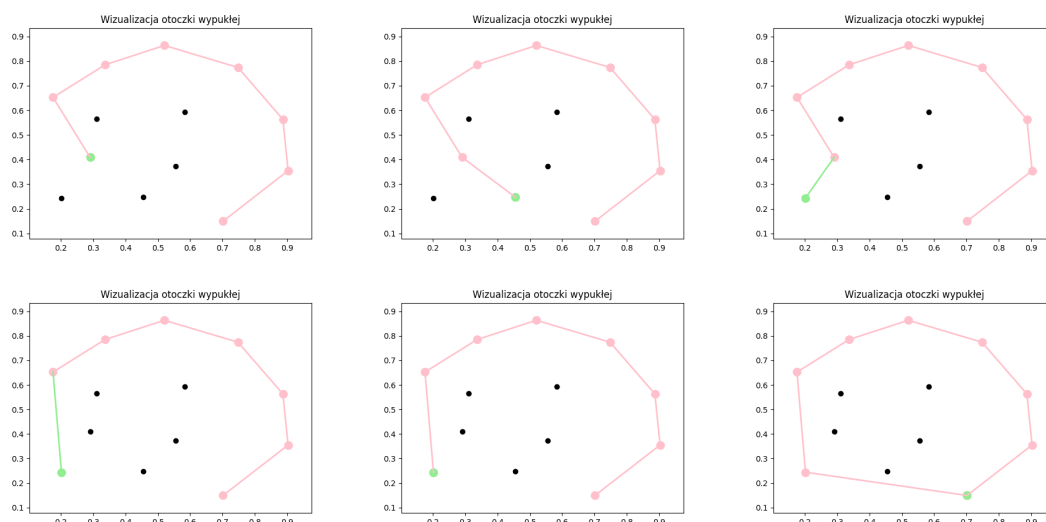
4.8.1 Działanie algorytmu

Algorytm Grahama, realizowany w przedstawionym kodzie, wyznacza otoczkę wypukłą zbioru punktów na płaszczyźnie poprzez kilka głównych etapów. Pierwszym krokiem jest wybór punktu bazowego q , który charakteryzuje się najmniejszą współrzędną y , a w przypadku remisu najmniejszą współrzędną x . Zadanie to realizuje funkcja `find_min_point`, przeszukując zbiór punktów Q w celu znalezienia odpowiedniego punktu. Następnie pozostałe punkty są przekształcane w listę krotek zawierających kąt względem osi x , odległość od punktu bazowego q , oraz współrzędne punktu p . Funkcja `set_angle_and_distance` oblicza kąty z użyciem `arccos` i odległości euklidesowe, umożliwiając dokładne sortowanie punktów.

Po sortowaniu, funkcja `delete_colinear` usuwa punkty, które leżą na jednej prostej (są współliniowe) względem punktu bazowego. Z pozostawionych punktów wybierany jest tylko ten, który znajduje się w największej odległości od punktu bazowego q , eliminując zbędne operacje na współliniowych punktach.

Na końcu, iteracyjne budowanie otoczki wypukłej realizowane jest w funkcji `graham_algorithm`. Najpierw trzy pierwsze punkty są dodawane do stosu S , a następnie kolejne punkty są sprawdzane pod kątem orientacji (czy tworzą zakręt w lewo). Funkcja `orient` oblicza orientację trzech kolejnych punktów na podstawie iloczynu wektorowego. W przypadku zakrętu w prawo, ostatni punkt jest usuwany ze stosu S . Algorytm kontynuuje do momentu, aż wszystkie punkty zostaną przeanalizowane, a stos S zawiera wierzchołki otoczki wypukłej.

4.8.2 Wizualizacja przebiegu algorytmu



Rysunek 7: Fragment przebiegu algorytmu Grahama dla losowoadanego zbioru punktów

4.8.3 Analiza złożoności obliczeniowej

Algorytm Grahama ma złożoność $O(n \log n)$, gdzie n to liczba punktów w otoczce. Operacją wymagającą największej złożoności czasowej jest sortowanie punktów względem kąta oraz odległości między każdym punktem a punktem z najmniejszą współrzędną y . Dzięki podejściu zaproponowanemu przez Grahama wszystkie pozostałe operacje (znajdowanie punktu z najmniejszą współrzędną y , wyznaczenie kątów oraz wyznaczenie finalnej otoczki) mają złożoność czasową $O(n)$. Złożoność tego algorytmu posiada zarówno wady, jak i zalety. Dużą zaletą jest jego stała załóżność - niezależnie od przygotowanego zestawu danych algorytm ten zawsze będzie działał w złożoności $O(n \log n)$. Wadą jest jednak fakt, że w przypadku niewielkiej liczbie punktów tworzących otoczkę algorytm ten będzie działał wolniej od np. algorytmu Jarvisa, którego złożoność czasową jest związana z liczbą punktów tworzących finalną otoczkę

4.9 Algorytm Jarvisa

Nazwa pliku z implementacją: `jarvis.ipynb`

4.9.1 Działanie algorytmu

Algorytm Jarvisa- gift wrapping algorithm, wyznacza otoczkę wypukłą zbioru punktów na płaszczyźnie poprzez iteracyjne wybieranie kolejnych punktów otoczki w porządku przeciwnym do ruchu wskazówek zegara. Pierwszym krokiem jest identyfikacja punktu bazowego, który ma najmniejszą współrzędną y , a w przypadku remisu najmniejszą współrzędną x . Realizuje to funkcja `min` z odpowiednią funkcją kluczową, która przeszukuje zbiór punktów `data_set` w czasie $O(n)$. Punkt ten inicjalizuje otoczkę `hull`, a zmienna `current` wskazuje na bieżący punkt otoczki.

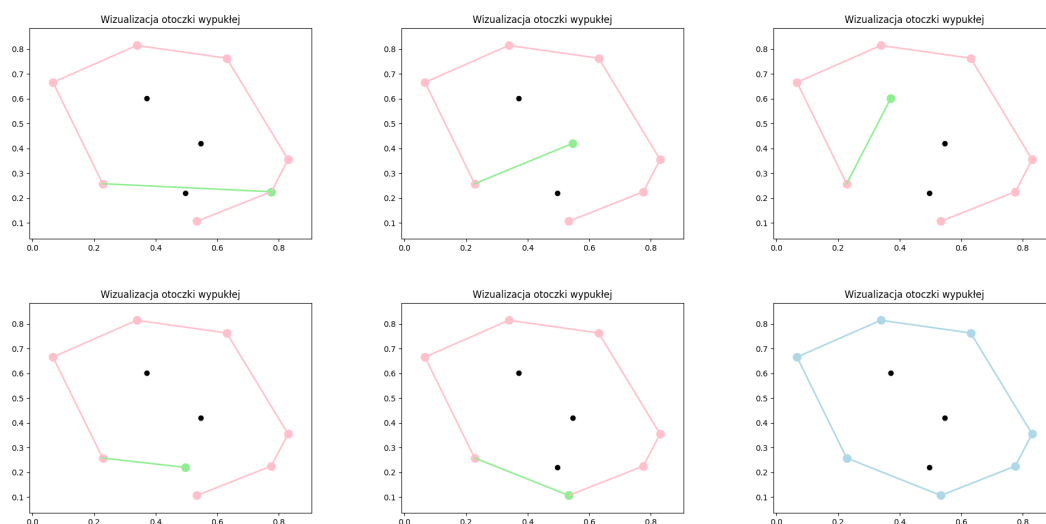
Dla każdego punktu otoczki algorytm wyszukuje następny punkt, iterując przez wszystkie pozostałe punkty zbioru `data_set`. W tym celu wykorzystuje funkcję `counter_clockwise`, która oblicza orientację trzech punktów na podstawie iloczynu wektorowego. Jeśli orientacja wskazuje zakręt przeciwny do ruchu wskazówek zegara (wartość ujemna), punkt staje się kandydatem na kolejny wierzchołek otoczki. W przypadku współliniowości wybierany jest punkt znajdujący się w większej odległości od bieżącego wierzchołka `current`.

Algorytm kontynuuje, aż punkt następny `next_point` będzie równy punktowi bazowemu, co oznacza, że otoczka wypukła została zamknięta. Wierzchołki otoczki są dodawane do listy `hull`, która po zakończeniu działania algorytmu zawiera wierzchołki otoczki w kolejności przeciwnie do ruchu wskazówek zegara.

4.9.2 Analiza złożoności obliczeniowej

Złożoność czasowa algorytmu Jarvisa wynosi $O(nh)$, gdzie n to liczba punktów w zbiorze wejściowym, a h to liczba punktów należących do otoczki wypukłej. Iteracyjne wyszukiwanie następnego punktu otoczki wymaga $O(n)$ operacji i jest powtarzane h razy, co czyni algorytm mniej wydajnym dla dużych zbiorów, gdy liczba punktów otoczki zbliża się do n .

4.9.3 Wizualizacja przebiegu algorytmu



Rysunek 8: Fragment przebiegu algorytmu Jarvisa dla losowo zadanego zbioru punktów

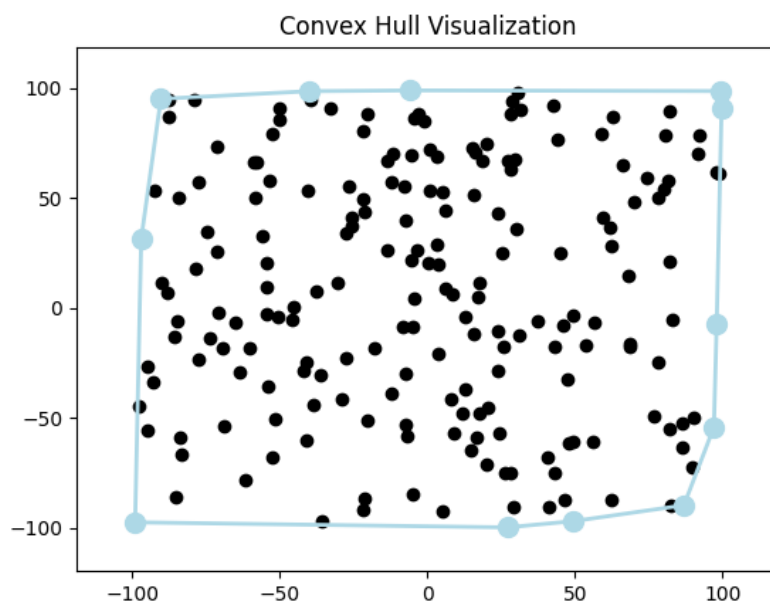
4.10 Testy algorytmów

W celu weryfikacji poprawności zaimplementowanych algorytmów przeprowadzono serię testów, które miały na celu zarówno ocenę poprawności ich działania, jak i szczegółową analizę ich właściwości. Badania te pozwoliły na identyfikację cech charakterystycznych algorytmów w różnych scenariuszach, w tym w kontekście danych o zróżnicowanych właściwościach. Przeanalizowano również zestawy danych sprzyjające oraz niekorzystne dla poszczególnych algorytmów, co umożliwiło pogłębioną ocenę ich mocnych i słabych stron. Tego typu analiza dostarcza istotnych informacji na temat efektywności algorytmów, ich ograniczeń oraz potencjalnych zastosowań w praktyce.

4.10.1 Zestaw 1: Losowy zestaw punktów

Aby zweryfikować poprawność działania algorytmu w standardowej sytuacji oraz ocenić czas jego działania, pierwszym krokiem jest analiza zestawu losowo wygenerowanych punktów. Tego rodzaju test umożliwia ocenę zarówno wydajności obliczeniowej, jak i poprawności działania algorytmu w warunkach ogólnych, niezależnych od specyficznych układów danych.

Losowy charakter danych pozwala również na przeprowadzenie eksperymentów symulujących typowe przypadki użycia, co czyni ten test wartościowym narzędziem w weryfikacji standardowej przydatności algorytmów.

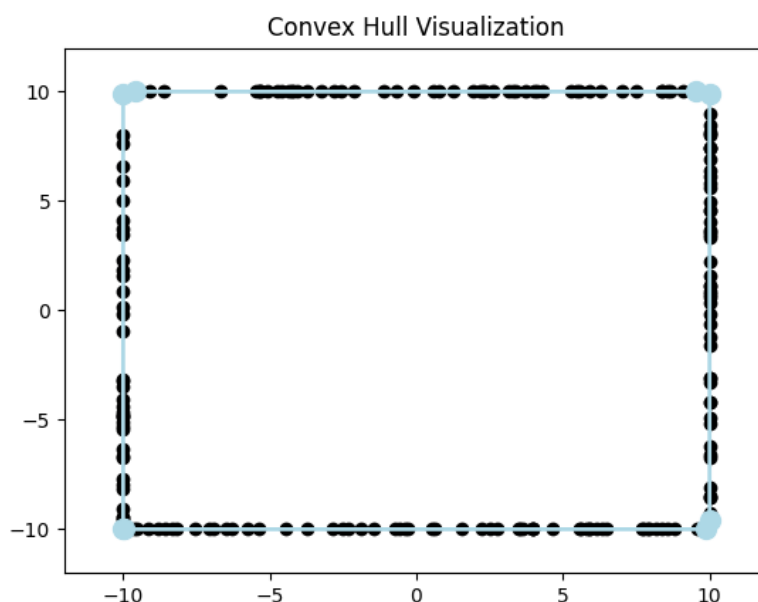


Rysunek 9: Otoczka wypukła dla zbioru losowych punktów

4.10.2 Zestaw 2: Zbiór punktów tworzących otoczkę 4-8 elementową

Aby zweryfikować działanie oraz różnice między algorytmami o złożoności $O(nh)$ i $O(n \log n)$, rozważany jest przypadek testowy obejmujący zestaw punktów, w których liczba elementów należących do otoczki wypukłej waha się w przedziale od 4 do 8. Tego typu konfiguracja umożliwia obserwację sytuacji, w której wyniki czasowe algorytmów są do siebie zbliżone.

Dzięki ograniczonej liczbie punktów tworzących otoczkę wypukłą, algorytmy o złożoności $O(nh)$, gdzie h jest liczbą punktów otoczki, mogą wykazywać zbliżoną wydajność do algorytmów $O(n \log n)$, które dominują przy większych wartościach h .

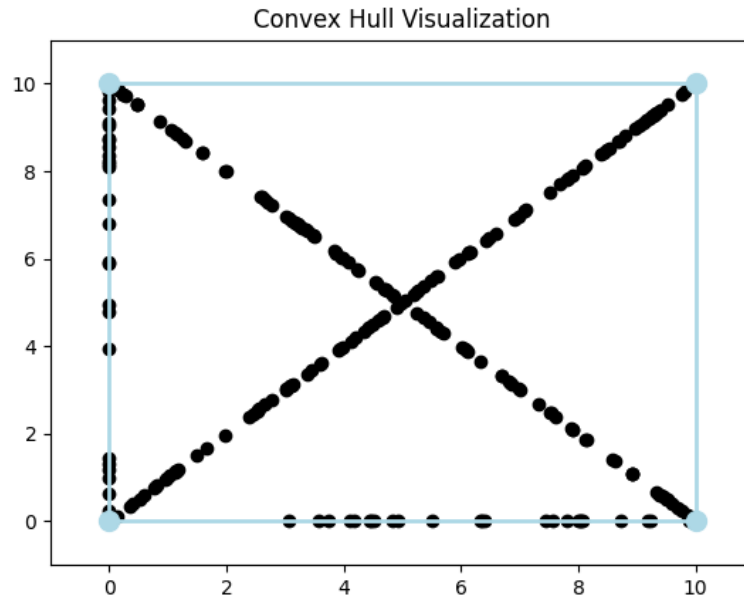


Rysunek 10: Otoczką wypukłą dla zbioru punktów leżących na planie prostokąta

4.10.3 Zestaw 3: Zbiór punktów tworzących otoczkę 4 elementową

Aby zweryfikować działanie oraz różnice między algorytmami o złożoności $O(nh)$, $O(n \log n)$ oraz $O(n^2)$, stworzono test z 4 punktami tworzącymi otoczkę. Test ten pozwala wykazać, które algorytmy wyznaczające otoczkę wypukłą wykazują najlepszą wydajność w przypadku małej liczby punktów tworzących otoczkę. Dzięki temu możliwe jest porównanie efektywności algorytmów, które różnią się podejściem do rozwiązania tego problemu w zależności od liczby punktów na otoczce.

Poza tym zbiór testowy może wskazać algorytmy, które nie wykorzystują tej właściwości, a w konsekwencji nie osiągają optymalnej wydajności w przypadkach z małą liczbą punktów na otoczce. Algorytmy te mogą wykazywać gorszą wydajność, ponieważ stosują generalną procedurę, która wymaga większej liczby operacji, niezależnie od liczby punktów na otoczce.

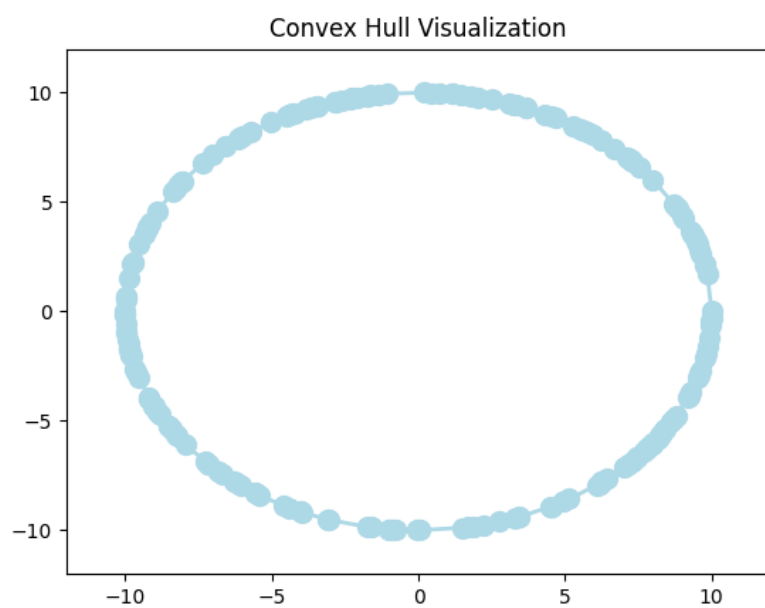


Rysunek 11: Otoczka wypukła dla zbioru punktów leżących na planie kwadratu oraz jego przekątnych wraz z dodatkowymi 4 punktami na wierzchołkach kwadratu.

4.10.4 Zestaw 4: Zbiór punktów na planie okręgu

Aby zweryfikować działanie algorytmów wyznaczających otoczkę wypukłą, warto rozważyć przypadek, w którym wszystkie punkty zbioru leżą na okręgu i tworzą jednocześnie otoczkę wypukłą. W takim przypadku każdy punkt jest częścią otoczki, ponieważ otoczką wypukłą dla zbioru punktów na okręgu to po prostu sam okrąg.

Zbiór taki pozwala na wykazanie różnic w wydajności algorytmów, które mogą różnie traktować sytuacje, w których wszystkie punkty są na granicy otoczki. Dzięki temu testowi możliwe jest zidentyfikowanie algorytmów, które są w stanie wykorzystać specyficzną strukturę zbioru punktów (punktów na okręgu) i osiągnąć lepszą wydajność w porównaniu do algorytmów, które nie uwzględniają tej szczególnej własności zbioru, co może prowadzić do niepotrzebnego zwiększenia liczby operacji.



Rysunek 11: Otoczka wypukła dla zbioru punktów leżących na planie okręgu

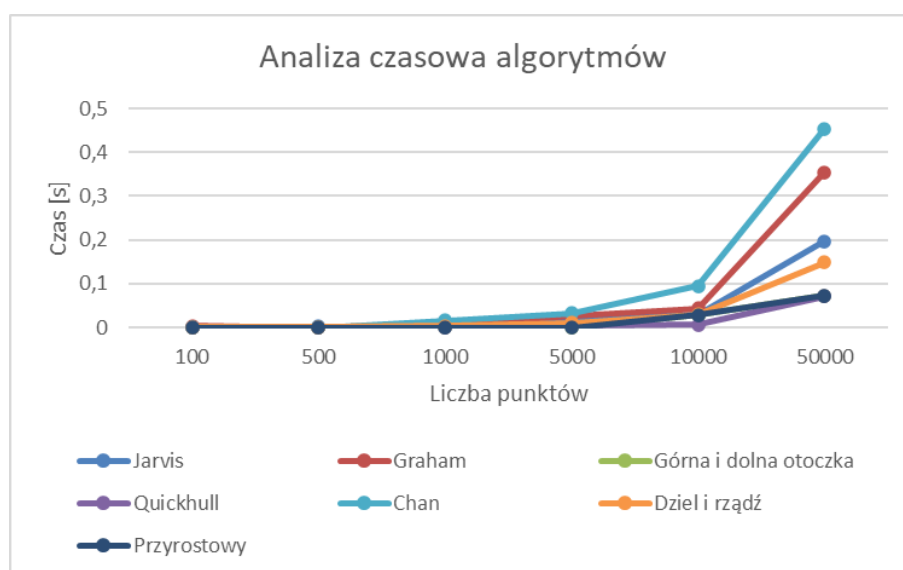
4.11 Wyniki czasowe działania algorytmów

4.11.1 Zestaw 1

Liczba punktów	Jarvis	Graham	Górna i dolna otoczka	Quickhull	Chan	Dziel i rządź	Przyrostowy
100	0.0	0.0035	0.0	0.0	0.0	0.001	0.0
500	0.0017	0.0	0.0	0.001	0.0	0.001	0.0
1000	0.0033	0.0078	0.0	0.001	0.0164	0.003	0.0
5000	0.0202	0.0273	0.0	0.005	0.0331	0.0112	0.0
10000	0.029	0.0442	0.028	0.0063	0.0947	0.0296	0.028
50000	0.197	0.3538	0.0721	0.0715	0.4535	0.1489	0.0721

Tabela 1: Czasy wykonania algorytmów w zależności od liczby punktów testowych w [s]

Wyniki danych przedstawionych w tabeli 1 zostały zaprezentowane na rysunku 12.



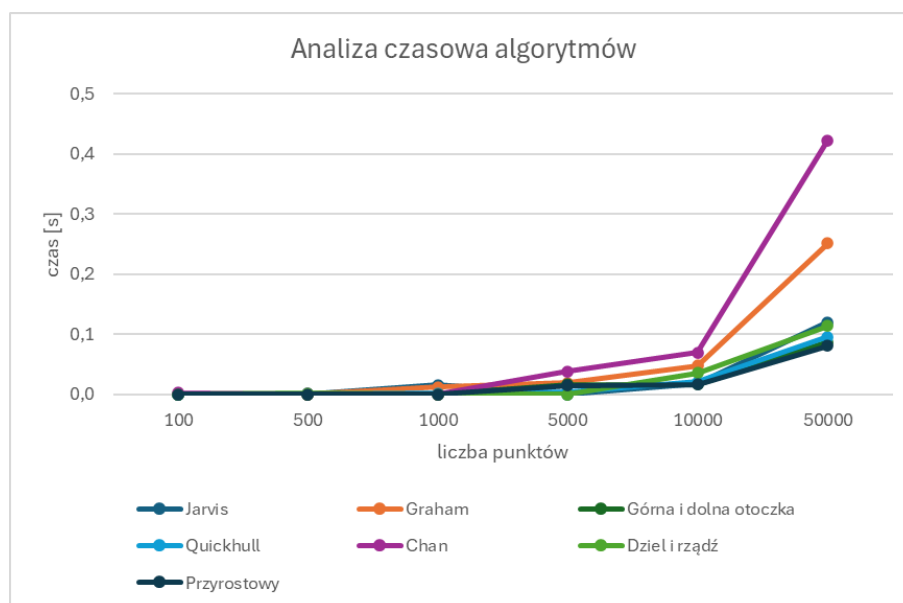
Rysunek 12: Wykres zależności czasu wykonania algorytmów od liczby punktów dla zestawu 1

4.11.2 Zestaw 2

Liczba punktów	Jarvis	Graham	Górna i dolna otoczka	Quickhull	Chan	Dziel i rządź	Przyrostowy
100	0.0	0.0	0.0	0.0	0.0026	0.0	0.0
500	0.0	0.0	0.0	0.001	0.0005	0.0017	0.0
1000	0.0159	0.0122	0.0	0.002	0.0	0.0	0.0
5000	0.0	0.0191	0.0163	0.004	0.0381	0.0	0.0144
10000	0.0184	0.0482	0.0167	0.0213	0.0692	0.0359	0.0163
50000	0.1205	0.2511	0.0872	0.0954	0.4228	0.1136	0.0813

Tabela 2: Czasy wykonania algorytmów w zależności od liczby punktów testowych w [s]

Wyniki danych przedstawionych w tabeli 2 zostały zaprezentowane na rysunku 13



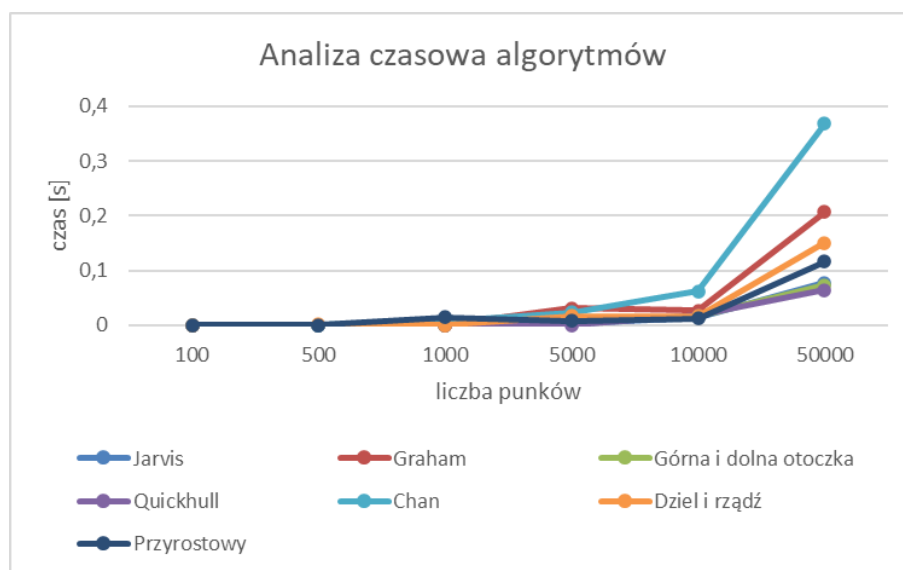
Rysunek 13: Wykres zależności czasu wykonania algorytmów od liczby punktów dla zestawu 2

4.11.3 Zestaw 3

Liczba punktów	Jarvis	Graham	Górna i dolna otoczka	Quickhull	Chan	Dziel i rządź	Przyrostowy
100	0.0	0.0	0.0	0.0	0.0	0.0	0.0
500	0.0	0.0	0.0	0.0	0.0	0.0011	0.0
1000	0.0022	0.0	0.0	0.0	0.0079	0.0	0.015
5000	0.006	0.0315	0.0067	0.0	0.0237	0.0166	0.008
10000	0.0147	0.0271	0.0161	0.0183	0.0625	0.017	0.0128
50000	0.078	0.2072	0.073	0.0647	0.3681	0.1511	0.1169

Tabela 3: Czasy wykonania algorytmów w zależności od liczby punktów testowych w [s]

Wyniki danych przedstawionych w tabeli 3 zostały zaprezentowane na rysunku 14



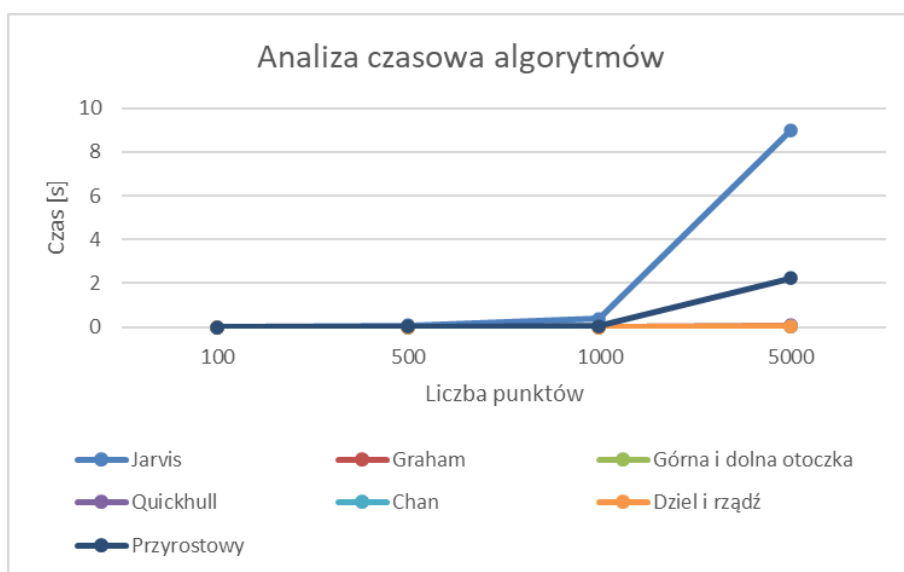
Rysunek 14: Wykres zależności czasu wykonania algorytmów od liczby punktów dla zestawu 3

4.11.4 Zestaw 4

Liczba punktów	Jarvis	Graham	Górna i dolna otoczka	Quickhull	Chan	Dziel i rządź	Przyrostowy
100	0.0012	0.0	0.0	0.0	0.0	0.0	0.0
500	0.0837	0.0	0.0	0.0167	0.0	0.013	0.0121
1000	0.3791	0.0	0.0	0.015	0.0156	0.0019	0.0213
5000	8.9774	0.0337	0.0168	0.0687	0.0156	0.0293	0.7661

Tabela 4: Czasy wykonania algorytmów w zależności od liczby punktów testowych w [s]

Wyniki danych przedstawionych w tabeli 4 zostały zaprezentowane na rysunku 15



Rysunek 15: Wykres zależności czasu wykonania algorytmów od liczby punktów dla zestawu 4

4.12 Wnioski i podsumowanie

Wszystkie testy przebiegły pomyślnie, a wyniki są zgodne z oczekiwaniami. Potwierdza to poprawność i rzetelność całego procesu.

4.12.1 Zbiór 1: Losowo rozmieszczone punkty

Najwolniejszy okazał się algorytm Chan'a (Rysunek 12), ponieważ jego implementacja w tym przypadku nie jest optymalna. Chociaż jego złożoność powinna wskazywać, iż będzie to najszybszy algorytm, z powodu losowej liczby punktów tworzących otoczkę w testach to on okazał się być najwolniejszy.

Z kolei najszybszy okazał się algorytm Quickhull, który skutecznie radzi sobie w sytuacjach, gdzie liczba punktów na otoczce wypukłej jest stosunkowo mała. W takich przypadkach heurystyki Quickhull minimalizują liczbę wykonywanych operacji. Podobny czas osiągnął algorytm przyrostowy. Jest to spowodowane stosunkowo niewielką liczbą punktów w finalnej otoczce. Reszta algorytmów wykazywała zbliżone czasy działania, co wskazuje na ich porównywalną efektywność w tym scenariuszu. Test ten pozwalał wykazać różnice głównie w sposobie implementacji, ze względu na zbiór znacząco nie faworyzujący żadnych algorytmów.

4.12.2 Zbiór 2: Zbiór z otoczką z 8 punktami

Przedstawione na rysunku 13 oraz w tabeli 2 dane pokazują, że najgorszy czas osiągnął algorytm Chan'a. Jest to spowodowane dużą liczbą otoczek, gdzie dla każdej z nich algorytm musi wyznaczyć punkt tworzący jak najmniejszy kąt z istniejącą już otoczką. Mimo, iż złożoność tego algorytmu jest najmniejsza ze wszystkich, to w praktyce nie zawsze jest to najszybszy sposób rozwiązania problemu.

Najszybsze okazały się algorytmy *górną i dolną otoczki* oraz *przyrostowy*. Dzięki stosunkowo niewielkiej liczbie punktów tworzących końcową otoczkę, algorytmy te były w stanie osiągnąć najlepsze rezultaty czasowe i zdecydowanie były najlepszym rozwiązaniem dla tego zbioru testów.

4.12.3 Zbiór 3: Zbiór z otoczką z 4 punktami

(Rysunek 15) W tym przypadku najgorzej wypadł algorytm Chan'a. Jego słaba wydajność wynika z dużej liczby podotoczek, gdzie dla każdej musi on odnaleźć najbardziej ekstremalny punkt co bardzo spowalnia jego działanie.

Quickhull ponownie był najszybszy, ponieważ efektywnie przetwarza przypadki, gdzie liczba punktów na otoczce jest mała. Reszta algorytmów wykazywała zbliżone czasy działania, co oznacza, że radzą sobie w podobny sposób w tym układzie punktów.

4.12.4 Zbiór 4: Wszystkie punkty należą do otoczki

Najgorzej wypadł algorytm Jarvisa, którego złożoność $O(nh)$ w tej sytuacji przekształca się w $O(n^2)$, ponieważ $h = n$. W przypadku, gdy wszystkie punkty należą do otoczki, każdy krok wymaga przetworzenia wszystkich punktów, co znacząco wydłuża czas działania.

Najlepiej poradził sobie algorytm Chan'a. Jego struktura, podział na mniejsze podgrupy, wyznaczeniu otoczki algorytmem Grahama a następnie łączenie w tym przypadku skraca się tylko do uruchomienia algorytmu Grahama. Z racji, iż wiemy, że każdy punkt należy do finalnej otoczki możemy podać dokładne m , a zatem uzyskać najlepszą złożoność.

Algorytm przyrostowy zajął relatywnie dużo czasu. Wynika to z jego iteracyjnego charakteru: dodaje on kolejne punkty i za każdym razem aktualizuje otoczkę. W przypadku, gdy wszystkie punkty należą do otoczki, staje się to bardzo czasochłonnym procesem.

Ten test kompleksowo weryfikował zdolność algorytmów do efektywnego wyznaczania otoczki wypukłej, gdy dominująca liczba punktów należy do tej otoczki.

4.12.5 Podsumowanie

Wyniki testów pokazują, że niezwykle istotną rzeczą jest odpowiedni dobór algorytmów do struktury danych wejściowych. Algorytm *Quickhull* wykazał największą uniwersalność, radząc sobie najlepiej w zbiorach 1 i 3. Algorytm *Chan'a* okazał się wyjątkowo skuteczny w zbiorze 4, gdzie wszystkie punkty należały do otoczki wypukłej. Z kolei algorytmy *Jarvisa* i *przyrostowy* wykazały wyraźne ograniczenia w przypadkach, gdzie ich złożoność teoretyczna osiąga swoje maksimum. Zadziwiające mogą być wyniki osiągnięte przez algorytm *Chan'a*. Mimo, iż jego złożoność jest najlepsza, to poprzez nieoptymalną aproksymację oraz możliwą implementację niespodziewanie przegrywa z innymi rozwiązaniami. Pozostałe algorytmy wykazywały się dobrymi rezultatami czasowymi, jednakże nie stanowiły one najbardziej optymalnych rozwiązań.

5 Bibliografia

1. Wikipedia, *Quickhull*, dostępne pod adresem: <https://pl.wikipedia.org/wiki/Quickhull> (Dostęp: 3 stycznia 2025)
2. Misha Kazhdan, *QuickHull Algorithm Lecture Notes*, dostępne pod adresem: <https://www.cs.jhu.edu/~misha/Spring16/07.pdf> (Dostęp: 3 stycznia 2025)
3. GeeksforGeeks, *Convex Hull using Divide and Conquer Algorithm*, dostępne pod adresem: <https://www.geeksforgeeks.org/convex-hull-using-divide-and-conquer-algorithm/> (Dostęp: 3 stycznia 2025)
4. Wikipedia, *Chan's Algorithm*, dostępne pod adresem: https://en.wikipedia.org/wiki/Chan%27s_algorithm (Dostęp: 3 stycznia 2025)
5. Pranay Yadav, *Repozytorium Convex-Hull*, dostępne pod adresem: <https://github.com/ypranay/Convex-Hull/blob/master/ChansAlgorithmForConvexHull.cpp> (Dostęp: 4 stycznia 2025)
6. Tom Switzer, *implementacja algorytmu Chan'a*, dostępne pod adresem: <https://gist.github.com/tixxit/252229> (Dostęp: 4 stycznia 2025)
7. Koło Naukowe Bit, *przykłady testowe*, dostępne pod adresem: <https://github.com/aghbit/Algorytmy-Geometryczne/blob/master/bitalg/lab2/main.ipynb> (Dostęp: 4 stycznia 2025)